

Descripción de Hapnow

Hapnow es una alternativa diseñada para facilitar la organización y descubrimiento de eventos espontáneos en tiempo real. A diferencia de otras aplicaciones de gestión de eventos tradicionales, Hapnow está enfocada en la inmediatez, permitiendo a los usuarios crear y unirse a actividades y eventos que ocurrirán, como máximo, en las próximas 12 horas.

La aplicación utiliza la geolocalización para conectar a personas con intereses comunes en una zona geográfica específica, mostrando eventos cercanos en un mapa interactivo. Los usuarios podrán filtrar eventos por categoría (deportes, ocio, estudio, actividades culturales, profesionales, etc.) y distancia, permitiendo una navegación intuitiva y directa sobre la oferta de actividades disponibles en su entorno inmediato.

Gracias a esta aplicación, se podría llegar a resolver de una forma muy solvente el problema de mucha gente, jóvenes sobre todo, respecto a socializar de una forma más dinámica. Existe una demanda creciente de herramientas que faciliten conexiones más auténticas con el mundo real, especialmente tras el impacto de la pandemia en los hábitos de socialización.

Esta aplicación no solo conecta diferentes usuarios de una zona para la realización de eventos, sino que construye un sistema de reputación basado en asistencia real verificada por check-in. Cada evento crea un espacio para comunicarse entre usuarios mediante un chat grupal donde los participantes pueden conocerse y coordinarse, y posteriormente, una galería colaborativa donde los asistentes pueden compartir fotos y videos del evento.

Justificación y alcance del proyecto

En la era en la que vivimos, en la era digital, cada vez son más las personas que experimentan soledad y dificultades para encontrar compañía para actividades cotidianas. Otras plataformas se enfocan en eventos planificados con comunidades ya establecidas, pero existe un vacío en el mercado enfocado en actividades espontáneas. Hapnow podría ser de gran ayuda para aquel estudiante que quiere practicar algún deporte con alguien por la tarde, o para un profesional que busca alguien para tomar un café ahora mismo.

El proyecto está diseñado para demostrar las competencias adquiridas durante el ciclo, integrando tecnologías modernas de desarrollo Frontend con Angular y servicios de arquitectura Cloud mediante Firebase. Al utilizar un modelo BaaS (Backend as a Service), el desarrollo se ha centrado en crear una interfaz sólida y reactiva, permitiendo al mismo tiempo profundizar en la gestión de bases de datos NoSQL en tiempo real, autenticación segura y almacenamiento en la nube, habilidades altamente demandadas en el mercado laboral actual.

Pese a plantearse como un proyecto académico, Hapnow tiene fundamentos para poder llegar a convertirse en una aplicación real. El modelo de negocio potencial incluiría, por

ejemplo, eventos privados, un mayor radio de búsqueda, analíticas para organizadores frecuentes y posibles patrocinios de negocios locales que aparecerían en eventos cercanos a sus ubicaciones. La arquitectura escalable de Firebase permitiría este crecimiento sin necesidad de una gran inversión inicial en infraestructura.

Valoración de alternativas existentes en el mercado

Eventbrite:

- Fortalezas: Muy buen software para grandes eventos, sistema de ticketing y herramientas de marketing.

Eventbrite es para organizadores profesionales. La creación de eventos en Hapnow está abierta para cualquier usuario de forma inmediata.

Bumble BFF:

- Fortalezas: Enfocado en hacer amistades, sistema de matching.

Estas aplicaciones requieren crear una conexión personal primero (sistema de matching), sin embargo con Hapnow conecta las personas mediante el propio evento en sí, siendo la actividad el punto de encuentro.

Facebook Events:

- Fortalezas: tiene una gran base de usuarios, integración en la propia red social y es gratuito.

Facebook Events no prioriza eventos locales inmediatos. Hapnow en su lugar, usa la geolocalización como eje central y filtra eventos de las próximas 12 horas.

Stack tecnológico

Frontend

- **Framework:** Angular 18+
- **Entorno:** VS Code
- **Lenguaje:** TypeScript
- **Estilos:** SCSS
- **Diseño:** Figma
- **Mapas Interactivos:** Leaflet
- **Geocolocación:** Nominatim (OpenStreetMap)

Backend

- **Base de Datos y Auth:** Firebase (Firestore & Auth)
- **Gestión Multimedia:** Cloudinary
- **Control de versiones:** GitHub

Objetivos y requisitos propuestos del sistema

En un principio, se decidieron estas características como requisitos a cumplir en el proyecto:

1. Implementar autenticación con Firebase Authentication.
2. Desarrollar mapa interactivo con eventos geolocalizados.
3. Crear sistema de chat en tiempo real.
4. Integrar notificaciones push.
5. Implementar filtrado por distancia, categoría y horario.
6. Desarrollar sistema de check-in geolocalizado.
7. Crear galería pos-evento con subida de fotos.
8. Implementar panel de administración.

Finalmente, todos los puntos se pudieron llegar a cabo salvo por la integración de notificaciones push y el filtrado por horario.

Requisitos funcionales finales

- **Autenticación:**
 - Gestión de perfiles de usuarios.
 - Recuperación de contraseña.
 - Registro y login con email/contraseña.
- **Gestión de eventos:**
 - Crear evento con ubicación, fecha, categoría y límite de participantes opcional.
 - Restricción temporal: 0-12 horas de antelación.
 - Editar/cancelar eventos propios.
 - Inscripción y desinscripción.
- **Búsqueda y Visualización:**
 - Mapa con marcadores.
 - Filtros: categoría y distancia (1-20km).
 - Detección automática de ubicación del usuario.
- **Chat Grupal:**
 - Acceso exclusivo para participantes inscritos.
 - Mensajería en tiempo real.
 - Moderación por usuario.

- **Check-in:**
 - Registro de asistencia.
- **Galería pos-evento:**
 - Subida de fotos y vídeos exclusiva para asistentes verificados.
- **Valoraciones:**
 - Puntuación de 1-5 estrellas para asistentes verificados.
- **Administración:**
 - Gestión de reportes.
 - Eliminación de contenido.
 - Suspensión de usuarios.
 - Estadísticas.

Requisitos de interfaz finales

- **Diseño visual:**
 - Paleta de SCSS.
 - Colores por categoría: Deportes (rojo), Ocio (azul), Estudio (verde), Cultural (morado), Profesional (naranja).
 - Tipografía Inter.
- **Navegación:**
 - Navbar: logo, buscador, notificaciones, perfil (logout), (icono admin*).
 - Botón para crear eventos y para ver eventos del usuario.
- **Responsive:**
 - Distribución en columna en móvil.

Casos de uso principales:

- Registrarse -> Usuario nuevo -> completa formulario -> se crea la cuenta -> redirige al dashboard.
- Crear evento -> usuario autenticado -> completa formulario de crear evento -> se crea el evento junto con la opción de chat
- Buscar Eventos -> usuario autenticado -> busca en la sección de eventos / en el mapa con marcadores -> ve detalles del evento
- Inscribirse a evento -> usuario autenticado -> ve detalles del evento -> (si no es el creador) hace check-in -> redirige al dashboard
- Compartir foto o video -> usuario con check-in -> hace click en input file -> sube foto o video -> se hace visible al terminar el evento
- Valorar evento -> usuario con check-in -> evento terminado -> selecciona estrellas (1-5) -> se actualiza reputación del organizador.
- Moderar contenido -> admin -> revisa reportes -> decide acción (eliminar/ignorar/suspender) -> se quita el reporte.

Modelo y diseño final de la BBDD

El modelo estaría basado en la estructura de Firebase, en colecciones, los cuales a su vez contienen sus correspondientes documentos, y dentro de cada uno puede llevar tambien colecciones:

- Eventos
 - id?: string
 - titulo: string
 - descripcion: string
 - categoría : 'deportes' | 'ocio' | 'estudio' | 'cultural' | 'profesional'
 - ubicacion
 - ciudad: string
 - direccion: string
 - fecha: Date
 - hora: string
 - participantes: string
 - maxParticipantes?: number
 - creadorNombre: string
 - creadorId: string
 - estado: 'activo' | 'cancelado' | 'finalizado'
 - fechaCreacion: Date
 - valoradoPor?: string[]
 - multimedia:
 - url: string
 - tipo: 'image' | 'video'
 - fecha: any
 - usuariold: string
 - usuarioNombre: string

Dentro de cada documento de evento, hay una coleccion llamada mensajes, que a su vez tiene sus documentos, y cada documento de mensaje tiene los siguientes campos:

- mensajes
 - id
 - fecha: Timestamp
 - texto: string
 - usuariold: string
 - usuarioNombre: string

Luego están los reportes:

- reportes
 - id
 - id?: string
 - usuarioReportadorId: string
 - usuarioReportadorNombre: string

- usuarioReportadold: string
- usuarioReportadoNombre: string
- tipoContenido: 'mensaje' | 'image' | 'video'
- contenidold: string
- eventold: string
- textoContenido?: string
- fechaReporte: any
- estado: 'pendiente' | 'revisado'

Y por ultimo usuarios:

- usuarios
 - uid
 - uid: string
 - email: string
 - nombre: string
 - suspendido?: boolean
 - reputacion: number
 - totalValoraciones: number
 - eventosCreados: number
 - eventosAsistidos: number
 - rol: 'usuario' | 'admin'
 - fechaRegistro: Date

Retos encontrados

- **Implementación del chat del evento:** tuve que buscar varias formas para ver cómo se podía incrustar dentro de un mismo evento, a parte de aplicarle las funciones de reporte.
- **Gestión de mapas interactivos:** puesto que no tengo practica con el uso de apis de ese estilo, por lo que tuve que recurrir a ayuda para implementarlo.
- **Conflictos de integración con Google Maps API:** tuve varios problemas relacionados primeramente con la api de Google, ya que me salian muchos conflictos con CORS y con temas de pagos.
- **Arquitectura de notificaciones:** intente implementar esta parte pero al final me salían demasiados errores tanto de código como de lógica mientras lo hacía, y teniendo el código ya tan avanzado preferí no hacerlo por miedo a fastidiar el producto final.
- **Gestión de estados en la moderación:** fue bastante complejo ver y entender cuál sería la estructura final de los reportes y de cómo se ejecutaban esas acciones.

Ampliaciones a futuro

1. Sistema de Notificaciones Push

Implementar avisos en tiempo real mediante Firebase Cloud Messaging (FCM). Esto permitiría notificar a los usuarios cuando se cree un evento de su interés cerca de su ubicación o cuando reciban mensajes nuevos en el chat, incluso si la aplicación no está abierta.

2. Perfiles de Usuario Avanzados

Desarrollar una sección de perfil detallada donde cada usuario pueda gestionar su **reputación (karma)**, ver el historial de eventos a los que ha asistido, sus medallas por participación y personalizar sus intereses para mejorar las recomendaciones del mapa.

3. Sistema de Check-in por QR

Para validar la asistencia real a los eventos y alimentar el sistema de reputación, se podría implementar un sistema de validación mediante códigos QR generados por el organizador en el momento del evento.

4. Algoritmo de Recomendación Inteligente

Integrar lógica de inteligencia artificial o filtrado colaborativo para sugerir eventos no solo por cercanía, sino por afinidad histórica del usuario, mejorando la tasa de éxito en las conexiones sociales.

5. Gestión de Grupos y Amigos

Añadir una capa social que permita seguir a otros usuarios u organizadores específicos, crear grupos de amigos permanentes y poder invitarlos directamente a nuevos eventos espontáneos.

Conclusiones

El desarrollo de Hapnow ha representado un desafío técnico significativo, principalmente por la adopción de un stack tecnológico avanzado y totalmente nuevo para mí. La profundización en Angular y la integración de Firebase como solución backend realmente me han acabado resultando muy útiles para llevar a cabo la aplicación de una forma más fácil de la que pensaba en un principio.

A lo largo del proceso, el mayor aprendizaje no ha sido solo el dominio de la sintaxis, sino la capacidad de investigación y resolución de problemas. La integración de diversas APIs externas (Leaflet, Nominatim, Cloudinary) presentó retos lógicos que requirieron un gran análisis y la búsqueda de soluciones ante las limitaciones encontradas.

Pese a todas las dificultades encontradas, tanto por las tecnologías nuevas, como por mi no tan sencilla capacidad de aprendizaje con respecto al código, he logrado comprender de muy buena forma finalmente como funciona cada una de las tecnologías y estoy orgulloso de poder entender un proyecto no tan simple y de verlo medianamente sencillo.